

Programação Avançada

Unidade 3: Técnicas de Programação

Aula 14: DataFrames e Vetorização

Prof. Eduardo Corrêa Gonçalves

17/06/2026

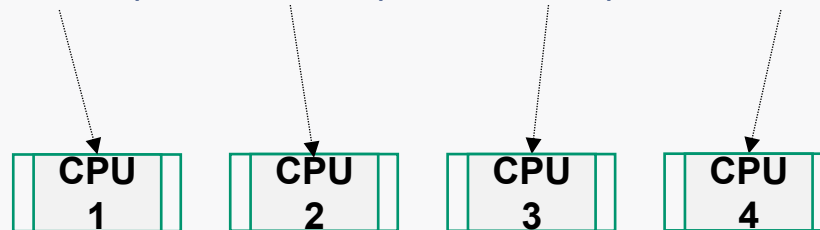
Sumário

- Introdução
 - O que é pré-processamento? O que é *data wrangling* ?
 - Data Frames
- Operações de data wrangling
 - Básicas: importação, fatiamento, projeção e filtragem
 - Exploração
 - Limpeza (*data cleaning*)
 - Transformação (*data transformation*)
- Notebook: data wrangling com a biblioteca pandas

Introdução (1/4)

- Na última aula – conhecemos a **programação paralela**
 - Forma de computação que possibilita com que **múltiplas cópias** de uma mesma funcionalidade sejam executadas ao mesmo tempo...
 - ... porém processando **dados diferentes**
 - ... Em CPUs diferentes

dados = [Cliente 1, Cliente 2, Cliente 3, Cliente 4, ...]



```
#TESTE 2: tempo de execução do map() paralelo
import multiprocessing

if __name__ == '__main__':

    with multiprocessing.Pool(processes=4) as pool:
        #executa pool.map()
        bd2 = tuple(pool.map(f_processar, clientes))
```

Introdução (2/4)

- **Hoje** – trabalharemos com a **vetorização**
 - Também permite com que uma mesma instrução (no caso, uma operação matemática) seja aplicada simultaneamente sobre dados diferentes.
 - Entretanto, apenas uma CPU é usada.
 - O efeito SIMD (Single Instruction Multiple Data) é obtido por meio de registradores estendidos (espécie de memória interna da CPU), muito maiores que os normais.
 - Eles permitem empacotar vários números (ex.: 8 inteiros) e, soma-los, dentre outras operações, em um único ciclo de clock
 - <https://datascience.blog.wzb.eu/2018/02/02/vectorization-and-parallelization-in-python-with-numpy-and-pandas/>

Introdução (3/4)

Data Frames

- Nessa aula, veremos como operações vetorizadas são aplicadas automaticamente sobre conjuntos de dados estruturados em **Data Frames**.

Mas o que é Data Frame ?

É a estrutura mais comum para representar **dados tabulares** em memória, ou seja, dados dispostos em **linhas** e **colunas**.

Não apenas no Python, mas também na linguagem R e outras

	<i>nome</i>	<i>continente</i>	<i>extensao</i>
AR	Argentina	América	2780
BR	Brasil	América	8511
FR	França	Europa	644
IT	Itália	Europa	301
UK	Reino Unido	Europa	244

df_paises

Introdução (4/4)

Características do Data Frames

- Disponibilizados pelo pacote '**pandas**' (*Python Data Analysis Library*) no Python.

Indispensável – esta é a estrutura mais importante lidar com bases de dados

Eficiente – internamente, faz uso de algoritmos otimizados (rápidos).

Facilita muito a **limpeza, transformação e exploração** de bases de dados.

	<i>nome</i>	<i>continente</i>	<i>extensao</i>
AR	Argentina	América	2780
BR	Brasil	América	8511
FR	França	Europa	644
IT	Itália	Europa	301
UK	Reino Unido	Europa	244

df_paises

Operações de Data Wrangling (1/2)

- Os Data Frames são usados principalmente para fazer *Data Wrangling*
 - Mesma coisa que pré-processamento...
 - São muitas operações
 1. Operações básicas: importação, fatiamento, projeção e filtragem
 2. Produção de estatísticas básicas (exploração dos dados)
 3. Limpeza e Transformação (*essas são as mais legais e importantes !*)

Operações de Data Wrangling (2/2)

- Primeiro passo – importação do pacote pandas

- Para trabalhar com DataFrames no Colab você precisa importar o pacote pandas com o comando **import**

import pandas as pd

- No Colab, basta colocar esse comando na 1ª célula de código
- Além de importar a biblioteca, a convenção é renomear para “pd” (por isso o “**as pd**”)
 - Motivo: dessa forma, dentro do código podemos usar o nome curto “pd” em vez de “pandas”

Operações Básicas (1/15)

Importação de dados

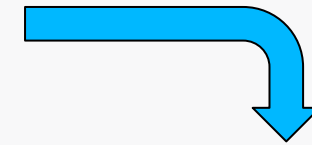
No cenário mais típico, os dados a serem importados para o data frame virão de **arquivo CSV**.

Vimos (na aula 3) que esse é o formato mais utilizado p/ armazenar BDs no formato tabular (dados dispostos em linhas e colunas).

Cada linha corresponde aos dados de um objeto (nesse exemplo, dados de um país)

E as colunas são separadas por vírgula.

```
países.csv - Bloco de Notas
Arquivo  Editar  Formatar  Exibir  Ajuda
sigla,nome,continente,extensao
AR,Argentina,América,2780
BR,Brasil,América,8511
FR,França,Europa,644
IT,Itália,Europa,301
UK,Reino Unido,Europa,244
Ln 7, Col 1  100%  Windows (CRLF)  UTF-8
```



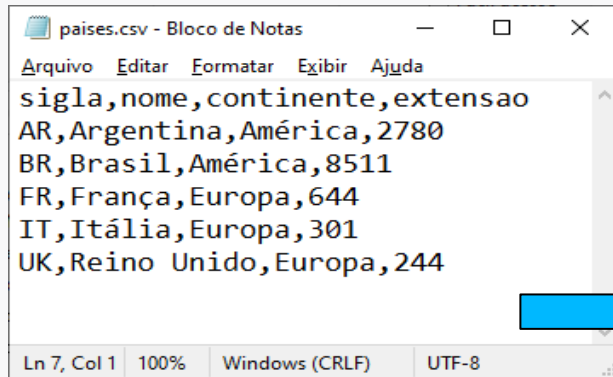
df_paises

	<i>nome</i>	<i>continente</i>	<i>extensao</i>
AR	Argentina	América	2780
BR	Brasil	América	8511
FR	França	Europa	644
IT	Itália	Europa	301
UK	Reino Unido	Europa	244

Operações Básicas (2/15)

• Importação de dados

- A principal função para importação de arquivos é **read_csv**.
- Se os dados do arquivo estão separados por vírgula e existe a linha do cabeçalho (formato padrão), basta indicar o caminho do arquivo.



- O parâmetro “index_col” é opcional. Ele é usado para que uma coluna se torne o rótulo das linhas (nesse caso, “sigla”).

- No **Colab** é possível **importar diretamente** arquivos que estão em um **endereço na Internet**.

- Se o arquivo estiver no seu Google Drive, também é possível, porém é preciso um código adicional

```
nome_arq = "https://raw.githubusercontent.com/edubd/bd/main/paises.csv"

df_paises = pd.read_csv(nome_arq, index_col = "sigla")
```

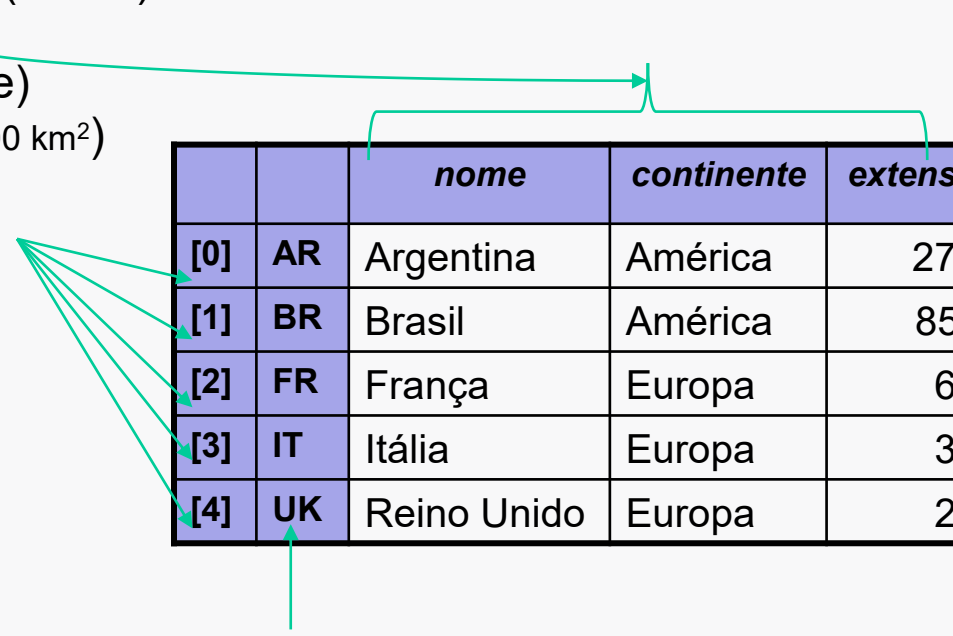
Operações Básicas (3/15)

- Características do Data Frame “df_países”, gerado pela importação

- **Colunas** possuem **nomes** ou **rótulos** (*labels*).

- “nome” (nome do país);
- “continente” (nome do continente)
- “extensão” (extensão territorial em 1000 km²)

- As **linhas** possuem **índices numéricos** de 0 até $n-1$ (onde n é o total de linhas).



		<i>nome</i>	<i>continente</i>	<i>extensao</i>
[0]	AR	Argentina	América	2780
[1]	BR	Brasil	América	8511
[2]	FR	França	Europa	644
[3]	IT	Itália	Europa	301
[4]	UK	Reino Unido	Europa	244

- Porém, as **linhas** podem também ser **opcionalmente rotuladas**
 - Nesse exemplo, foram rotuladas com “sigla do país”.

Operações Básicas (4/15)

- **Consultando as propriedades**

- Dado um DataFrame chamado "df", podemos usar os seguintes comandos para consultar suas propriedades básicas:
- **df.shape**: retorna o número de linhas e colunas
- **df.shape[0]**: número de linhas
- **df.shape[1]**: número de colunas
- **df.index**: índices das linhas (podem estar ou não rotulados)
- **df.columns**: nomes (ou rótulos) das colunas

```
print('shape (num. linhas e cols):', df_paises.shape)
print('número de linhas:', df_paises.shape[0])
print('número de colunas:', df_paises.shape[1])
print('índices das linhas:', df_paises.index)
print('nomes das colunas:', df_paises.columns)
```



```
shape (num. linhas e cols): (5, 3)
número de linhas: 5
número de colunas: 3
índices das linhas: Index(['AR', 'BR', 'FR', 'IT', 'UK'], dtype='object', name='sigla')
nomes das colunas: Index(['nome', 'continente', 'extensao'], dtype='object')
```

Operações Básicas (5/15)

- **Projeção e Filtragem**: as operações de Data Wrangling mais elementares
 - **Projeção**: operação que gera um novo DataFrame a partir da extração de algumas colunas de um outro.
 - Útil para **descartar atributos** que não precisarão ser envolvidos no seu modelo ou estudo estatístico.
 - **Filtragem**: operação que gera um novo DataFrame a partir da seleção (ou filtragem) de algumas linhas de interesse.
 - É chamada de filtragem porque visa **filtrar as linhas desejadas** para a criação do modelo.
 - *As duas operações são operações do tipo “fatiamento”, pois servem para selecionar uma “fatia” do DataFrame. Existem outras operações de fatiamento, mas não cobriremos no curso*

Operações Básicas (6/15)

- **Projeção:** basta passar uma lista com os nomes das colunas (atributos) de interesse.
 - Nenhuma linha é descartada. Apenas colunas são removidas.
- **Sintaxe:** indicar o nome do data frame, depois colchetes e dentro deles colocar a lista de colunas que será projetada (veja que a lista também é especificada com colchetes)

```
# gera um novo DataFrame apenas com as extensão e continente  
# de cada país (nessa ordem)  
df_cont_ext = df_paises[['extensao', 'continente']]  
print(df_cont_ext)
```

	extensao	continente
AR	2780	América
BR	8511	América
FR	644	Europa
IT	301	Europa
UK	244	Europa

Operações Básicas (7/15)

- **Projeção (exemplo 2):** se você quiser usar as posições das colunas em vez dos rótulos, é preciso usar a função **iloc**.
 - **Sintaxe:** `df.iloc[:, [lista_de_posições_de_colunas]]`
 - “:” no primeiro parâmetro significa “todas as linhas”

```
# projeção passando lista com posições das colunas - método iloc
# nesse exemplo, criamos um novo dataframe com as cols 2
(extensão) e 0 (nome)
df_ext_nome = df_países.iloc[:, [2, 0]]
print("Projeção 2: usando iloc")
print(df_ext_nome)
```

```
Projeção 2: usando iloc
      extensão      nome
AR         2780  Argentina
BR         8511   Brasil
FR          644   França
IT          301   Itália
UK          244 Reino Unido
```

Operações Básicas (8/15)

- **Filtragem:** normalmente implementada por **indexação booleana**.
 - Teste sobre uma ou mais colunas que resultará em verdadeiro ou falso para cada linha. Apenas as linhas onde o teste é verdadeiro são mantidas.
 - Nenhuma coluna é descartada. Apenas linhas são removidas.
- **Sintaxe:** indicar o nome do data frame, depois colchetes e dentro deles colocar o filtro. O filtro pode ser montado antes (como fiz no exemplo), usando parênteses.

```
# gera novo DataFrame apenas com países c/ extensão acima de 1000
filtro = (df_paises['extensao'] > 1000)
df_grandes = df_paises[filtro]
print(df_grandes)
```

	nome	continente	extensao
AR	Argentina	América	2780
BR	Brasil	América	8511

Operações Básicas (9/15)

- **Filtragem – Exemplo 2:** filtro composto por duas condições
 - Nesse caso, cada condição deve estar entre parênteses
 - Podemos combinar as condições com o uso dos operadores lógicos | (ou), & (and) e ! (not)

```
# exemplo 2 - países da Europa e com extensão abaixo de 500
# É um teste com 2 condições combinadas pelo operador lógico & (and)
filtro2 = (df_países['continente'] == 'Europa') &
(df_países['extensao'] < 500)
df_menores_europa = df_países[filtro2]
print("Filtragem 2: duas condições")
print(df_menores_europa)
```

	nome	continente	extensao
IT	Itália	Europa	301
UK	Reino Unido	Europa	244

Operações Básicas (10/15)

- **Combinando a filtragem e a projeção**
 - Para combinar a filtragem e a projeção em uma única operação, você pode usar a função **loc**.
 - **Sintaxe:** `df.loc[filtro, [lista_de_colunas]]`
 - No exemplo abaixo, projeta-se o “nome” dos países da América (filtro)

```
# Exemplo 3 - combinando projeção e filtragem
# Retornando apenas o nome (projeção) dos países da América (filtro)
filtro = (df_países['continente'] == 'América')
df_america = df_países.loc[filtro, ['nome']]
print(df_america)
```

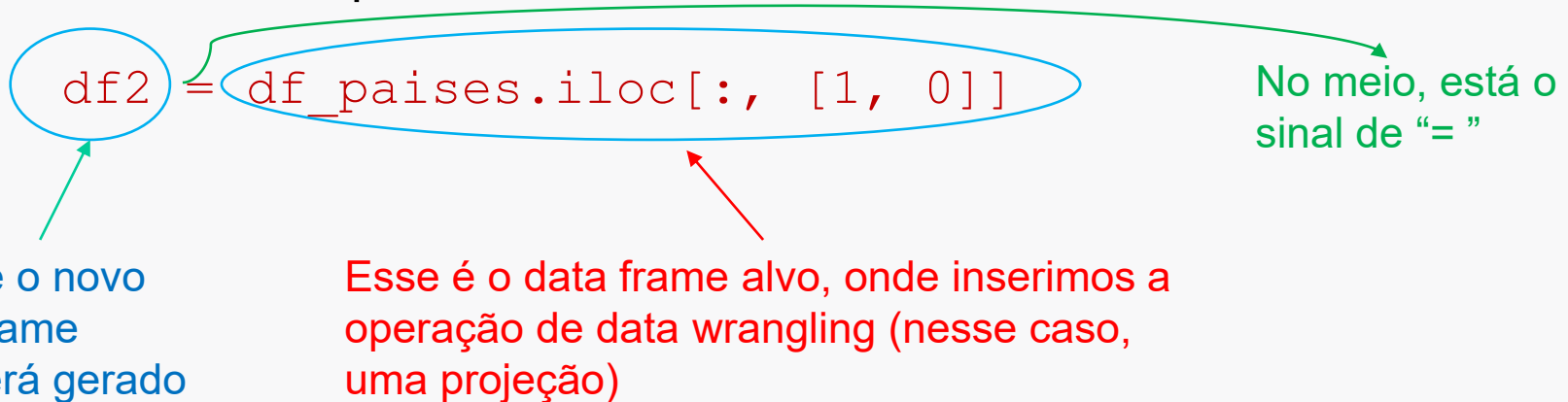
Exemplo 3: Retornando apenas o nome (projeção) dos países da América (filtro)

sigla	nome
AR	Argentina
BR	Brasil

Operações Básicas (11/15)

- **IMPORTANTE:** implementação de operações de data wrangling

- Uma operação de data wrangling funciona da seguinte forma:
 - Tem por **alvo** um **data frame existente**.
 - Produz como **resultado** um **novo data frame**, que consiste em uma **versão transformada** do data frame alvo.
- Nos programas, adota-se a sintaxe de atribuição (usando o sinal de “=”), como no exemplo abaixo:



Operações Básicas (12/15)

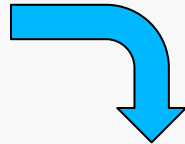
- **IMPORTANTE (2) – um MISTÉRIO:** quando é **clone**, quando é **visão** ??
 - Normalmente, as operações de data wrangling geram **novos data frames**, mas **isso não é garantido** (depende da natureza do data frame alvo da operação)
 - Às vezes não gera uma cópia, mas sim uma **visão** para a fatia do DataFrame original.
 - Isso é ruim, pois se você alterar a visão, isso será refletido no DataFrame original !
 - Infelizmente, as regras são bem confusas. Há uma explicação em: https://www.practicaldatascience.org/html/views_and_copies_in_pandas.html
 - No entanto, é possível forçar com que o fatiamento seja retornado em um **novos data frame** acrescentando **.copy()** no final do comando:

```
novo_df = df_paises[['extensao', 'continente']].copy()
```


Operações Básicas (14/15)

- Importação de planilha Excel
 - Para importar uma planilha Excel, utilize a função **read_excel**.

capital	região	população
Belém	Sudeste	1446042
Belo Horizonte	Sudeste	2513451
Boa Vista	Norte	326419
Curitiba	Sul	1893977
Florianópolis	Sul	477798
Macapá	Norte	465495
Manaus	Norte	2094391
Palmas	Norte	279856
Porto Alegre	Sul	1481019
Porto Velho	Norte	511219
Rio Branco	Norte	377057
São Paulo	Sudeste	12038175
Rio de Janeiro	Sudeste	6498837
Vitória	Sudeste	359555



```
planilha = "https://github.com/edubd/bd/blob/main/capitais.xlsx?raw=true"  
df = pd.read_excel(planilha)
```

Operações Básicas (15/15)

• Importando um arquivo do seu google drive

- Nesse caso, primeiro o Colab te obriga a colocar uma célula de código para montar o drive

- Com isso, ele passará a ficar visível no seu notebook

- Mas ao rodar o código, você terá que dar permissão de acesso

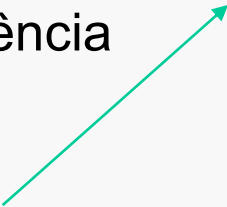
```
# para poder acessar arquivos do seu google drive,  
# é preciso rodar o código abaixo que "monta o drive"  
# e dar permissão de acesso  
from google.colab import drive  
drive.mount('/content/drive')
```

- Depois disso, basta indicar o caminho até o seu arquivo.
 - Abaixo um exemplo hipotético que abre o arquivo “produtos.csv” na pasta “bases” do Google Drive da pessoa.
 - Não adianta tentar rodar, pois só funciona se a pasta e arquivo existirem no seu drive

```
caminho = '/content/drive/My Drive/bases/produtos.csv'  
df = pd.read_csv(caminho)
```

Explorando os dados (1/7)

- **Ex. 1: características de atributos categóricos e discretos**
 - **unique()**: retorna um array com o domínio de um atributo – todas as categorias ou valores distintos que ele possui
 - **value_counts()**: vai além do método anterior, retornando uma Series a frequência de cada categoria ou valor.
- Vamos exemplificar usando a base de dados “lojas.csv” (ao lado)
 - Possui “raiz do cnpj”, “sufixo do cnpj”, “nome fantasia”, “total de pessoas ocupadas” (po), gasto com salários (salario) e “uf” de diversas empresas.



raiz	sufixo	fantasia	po	salario	uf
22222222	0001	PYTHONSHOP I	20	35000	SP
22222222	0002	PYTHONSHOP II	5	4000	RJ
22222222	0003	PYTHONSHOP III	15	31000	MG
33333333	0001	NPSERV LOJA A	8	9200	MG
33333333	0002	NPSERV LOJA B	0	0	MG
44444444	0001	XYZ MATRIZ	112	250000	RJ
44444444	0003	XYZ CENTRO	101	103900	SP
44444444	0004	XYZ NORTE	48	60000	RJ
44444444	0005	XYZ SUL	50	65000	RJ
44444444	0010	XYZ LITORAL	1	2500	RJ
55555555	0001	JAVASERV	40	40000	SP
66666666	0001	CAFEDB	12	12900	RS
99999999	0001	PANDAS KING	1	3000	MG

Explorando os dados (2/7)

- Ex. 1: características de atributos categóricos e discretos**

```
# unique -> para identificar o domínio (valores distintos) de um atributo
# value_counts -> para obter a frequência de cada valor no DataFrame
df_lojas =
pd.read_csv("https://raw.githubusercontent.com/edubd/bd/main/lojas.csv")

uf = df_lojas['uf'] # view para a coluna UF

print('- domínio: ', uf.unique())
print('- total de valores distintos: ', len(uf.unique()))
print('- frequência: '); print(uf.value_counts())
```

```
atributo "uf"
- domínio: ['SP' 'RJ' 'MG' 'RS']
- total de valores distintos: 4
- frequência:
uf
RJ 5
MG 4
SP 3
RS 1
Name: count, dtype: int64
```

Explorando os dados (3/7)

- **Ex. 2: estatísticas básicas para variáveis numéricas**
 - Para variáveis numéricas, podemos obter as medidas de tendência central e variabilidade.

```
po = df_lojas['po'] # view para a coluna po (pessoal ocupado)

print('- média: ', po.mean())
print('- mediana: ', po.median())
print('- mínimo: ', po.min(), ' - máximo: ', po.max())
print('- moda: ', po.mode().values)
print('- desvio padrão: ', po.std())
```

```
- média: 31.76923076923077
- mediana: 15.0
- mínimo: 0 - máximo: 112
- moda: [1]
- desvio padrão: 37.514783410796525
```

Explorando os dados (4/7)

• Ex. 3: estatísticas básicas para variáveis numéricas

- Assim como no R, podemos usar o método **describe()** para obter as estatísticas de todas as variáveis numéricas de uma vez só.
- Mas veja no exemplo abaixo que a pandas “entendeu” que ‘cnpj’ e ‘sufixo’ são numéricas.
 - Embora armazenem números, elas são nominais...

```
df_lojas.describe()
```

	raiz	sufixo	po	salario
count	1.300000e+01	13.000000	13.000000	13.000000
mean	4.444444e+07	2.692308	31.769231	47423.076923
std	2.127616e+07	2.562050	37.514783	68283.833541
min	2.222222e+07	1.000000	0.000000	0.000000
25%	3.333333e+07	1.000000	5.000000	4000.000000
50%	4.444444e+07	2.000000	15.000000	31000.000000
75%	4.444444e+07	3.000000	48.000000	60000.000000
max	1.000000e+08	10.000000	112.000000	250000.000000

Explorando os dados (5/7)

- Como corrigir?

- Para corrigir, basta forçar o tipo das colunas na importação.
- No `read_csv()`, use o parâmetro `dtype` e especifique um dicionário com os tipos desejados para as colunas escolhidas.

```
nome_arq = "https://raw.githubusercontent.com/edubd/bd/main/lojas.csv"
df_lojas = pd.read_csv(nome_arq,
                        dtype={"raiz": str, "sufixo": str})
print(df_lojas.dtypes)
print(df_lojas.describe())
```

raiz	object
sufixo	object
fantasia	object
po	int64
salario	int64
uf	object
dtype:	object

	po	salario
count	13.000000	13.000000
mean	31.769231	47423.076923
std	37.514783	68283.833541
min	0.000000	0.000000
25%	5.000000	4000.000000
50%	15.000000	31000.000000
75%	48.000000	60000.000000
max	112.000000	250000.000000

Explorando os dados (6/7)

- **Ex. 3: groupby**

- Permite computar estatísticas para grupos de linhas No exemplo abaixo, estatísticas de gastos com salário por UF
 - 'uf' é a chave do grupo (variável categórica)
 - Podemos utilizar um ou mais atributos na chave.

```
# PASSO1: criar o objeto group by
grp_salario_uf = df_lojas['salario'].groupby(df_lojas['uf'])

# PASSO 2: agora é só gerar as tabulações
print('- qtd de lojas por UF', grp_salario_uf.count())
print('- soma gasto com sal. p/ UF', grp_salario_uf.sum())
print('- média gasto com sal. p/ UF', grp_salario_uf.mean())
```

Explorando os dados (7/7)

- Ex. 3: `groupby()`

```
- quantidade de lojas por UF uf
```

```
MG      4
```

```
RJ      5
```

```
RS      1
```

```
SP      3
```

```
Name: salario, dtype: int64
```

```
- soma do gasto com salário por UF uf
```

```
MG      43200
```

```
RJ     381500
```

```
RS      12900
```

```
SP     178900
```

```
Name: salario, dtype: int64
```

```
- média do gasto com salário por UF uf
```

```
MG     10800.000000
```

```
RJ     76300.000000
```

```
RS     12900.000000
```

```
SP     59633.333333
```

```
Name: salario, dtype: float64
```

Limpeza de Dados (1/6)

- O objetivo da **limpeza** de dados é:
 - Eliminar **sujeira** e informações irrelevantes do BD.
- **Exemplos:**
 - Remover valores duplicados.
 - Remover ou modificar valores nulos.
 - Remover ou tratar outliers (valores discrepantes).

<i>id</i>	<i>nome</i>	<i>endereco</i>	<i>idade</i>	<i>sexo</i>	<i>salario</i>
4003	Joaquim R. Souza	Av. Alfa, 500	45	M	6500
4006	Ana Maria	Rua do Sol, 1234	32	F	NaN
4009	Joaquim R. Souza	Av. Alfa, 500	45	M	6500
4015	Antônia Silva	Rua ABC, 99	23	F	2000
4027	Glauco Filho	Av. Utopia s/n	1992	M	3000

Limpeza de Dados (2/6)

- **Use o código abaixo para criar o DataFrame do exemplo:**
 - Nesse exemplo, estamos criando o DataFrame de forma “hard-coded” (dentro do código) em vez de importar de um arquivo CSV.
 - Você pode usar esse recurso sempre que você quiser fazer algum teste...

```
import pandas as pd

dados = {
    "nome": ['Joaquim R. Souza', 'Ana Maria', 'Joaquim R. Souza',
            'Antônia Silva', 'Glauco Filho'],
    "endereco": ['Av. Alfa, 500', 'Rua do Sol, 1234', 'Av. Alfa, 500',
                'Rua ABC 99', 'Av. Utopia s/n'],
    "idade": [45, 32, 45, 23, 1990],
    "salario": [6500, None, 6500, 2000, 3000]
}

ids = [4003, 4006, 4009, 4015, 4027]

df_clientes = pd.DataFrame(dados, index=ids)
df_clientes.index.name = 'id'
print(df_clientes)
```


Limpeza de Dados (3/6)

- **Exemplo 1: removendo linhas duplicadas**

- Podemos usar a função **drop_duplicates()**.
- Por padrão faz a comparação considerando todas as colunas. Mas podemos especificar as colunas de interesse passando uma lista no parâmetro **subset**.
- https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.drop_duplicates.html

```
#mantém apenas a 1a linha do Joaquim  
df_clientes = df_clientes.drop_duplicates()  
print(df_clientes)
```



	nome	endereço	idade	salário
id				
4003	Joaquim R. Souza	Av. Alfa, 500	45	6500.0
4006	Ana Maria	Rua do Sol, 1234	32	NaN
4009	Joaquim R. Souza	Av. Alfa, 500	45	6500.0
4015	Antônia Silva	Rua ABC 99	23	2000.0
4027	Glauco Filho	Av. Utopia s/n	1990	3000.0

Limpeza de Dados (4/6)

- **Exemplo 2: tratamento de nulos abordagem 1 – remover linha**
 - Se o objetivo é remover as linhas com nulo, basta fazer uma filtragem + atribuição

```
filtro = df_clientes['salario'].notnull()

print(df_clientes[filtro])

# para alterar o DataFrame de verdade,
# é preciso fazer uma atribuição (deixei comentado)

# df_clientes = df_clientes[filtro]
```



id	nome	endereco	idade	salario
4003	Joaquim R. Souza	Av. Alfa, 500	45	6500.0
4006	Ana Maria	Rua do Sol, 1234	32	NaN
4015	Antônia Silva	Rua ABC 99	23	2000.0
4027	Glauco Filho	Av. Utopia s/n	1990	3000.0

Limpeza de Dados (5/6)

- **Exemplo 3: tratamento de nulos abordagem 2 – imputação**
 - Podemos trocar o valor nulo por um outro (ex.: valor médio, moda, ...)
 - Basta usar o método **fillna()** – troca nulos por outro valor
 - <https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.fillna.html>

```
sal_medio = df_clientes['salario'].mean() #  
média salários  
print('sal. medio = ', sal_medio)  
  
df_clientes['salario'] = df_clientes['salario'].fillna  
a(sal_medio)  
print(df_clientes)
```



```
sal. medio = 3833.3333333333335  


|      | nome             | endereco         | idade | salario     |
|------|------------------|------------------|-------|-------------|
| id   |                  |                  |       |             |
| 4003 | Joaquim R. Souza | Av. Alfa, 500    | 45    | 6500.000000 |
| 4006 | Ana Maria        | Rua do Sol, 1234 | 32    | 3833.333333 |
| 4015 | Antônia Silva    | Rua ABC 99       | 23    | 2000.000000 |
| 4027 | Glauco Filho     | Av. Utopia s/n   | 1990  | 3000.000000 |


```

Limpeza de Dados (6/6)

- **Exemplo 4: modificar um valor**

- Podemos usar a função **loc()** para fazer uma atualização de valor com base em uma condição.
- **Ex.:** trocar a idade 1990 para o valor correto (vamos supor que é 31)

```
#1. cria um filtro para localizar o cliente com idade = 1990
filtro = (df_clientes['idade']==1990)

#2. usa o loc com esse filtro para mudar a idade para 31
df_clientes.loc[filtro, 'idade']=31

print(df_clientes)
```



id	nome	endereço	idade	salário
4003	Joaquim R. Souza	Av. Alfa, 500	45	6500.000000
4006	Ana Maria	Rua do Sol, 1234	32	3833.333333
4015	Antônia Silva	Rua ABC 99	23	2000.000000
4027	Glauco Filho	Av. Utopia s/n	31	3000.000000

Transformação de Dados (1/13)

- O objetivo da **transformação** de dados é:
 - **Converter** os dados de origem para algum outro formato, mais adequado para ser utilizado por um algoritmo ou método estatístico.
- **Exemplos:**
 - Transformar uma variáveis para um formato mais adequado.
 - **Ex1.:** transformar “data de nascimento” em “idade”.
 - **Ex2.:** transformar “salário” em “faixa salarial”.
 - Normalização de dados.
 - **Ex.:** mapear todos os valores das variáveis numéricas para um número entre 0 e 1.
 - Gerar uma variável derivada de 2 ou mais variáveis do BD original.
 - **Ex.:** gerar a variável “renda_familiar” através da soma das variáveis “renda_conjuge1” e “renda_conjuge2”.

Transformação de Dados (2/13)

- Utilizaremos a base **flags.csv** para exemplificar algumas técnicas de transformação
 - Armazena informações sobre países e suas bandeiras* (dados dos anos 1980s...)

```
import pandas as pd

caminho = 'https://raw.githubusercontent.com/edubd/bd/main/flags.csv'
flags = pd.read_csv(caminho)

print(flags.head());
print(flags.tail())
```

* <https://archive.ics.uci.edu/ml/datasets/Flags>

Transformação de Dados (3/13)

- Base de dados **flags.csv** (um trecho dela...)

1	name	landmass	zone	area	population	language	religion	bars	stripes	colours	red	green	blue	gold	white	black	orange	mainhue	circles	c
2	Afghanistan	5	1	648	16	10	2	0	3	5	1	1	0	1	1	1	0	green	0	0
3	Albania	3	1	29	3	6	6	0	0	3	1	0	0	1	0	1	0	red	0	0
4	Algeria	4	1	2388	20	8	2	2	0	3	1	1	0	0	1	0	0	green	0	0
5	American-Samoa	6	3	0	0	1	1	0	0	5	1	0	1	1	1	0	1	blue	0	0
6	Andorra	3	1	0	0	6	0	3	0	3	1	0	1	1	0	0	0	gold	0	0
7	Angola	4	2	1247	7	10	5	0	2	3	1	0	0	1	0	1	0	red	0	0
8	Anguilla	1	4	0	0	1	1	0	1	3	0	0	1	0	1	0	1	white	0	0
9	Antigua-Barbuda	1	4	0	0	1	1	0	1	5	1	0	1	1	1	1	0	red	0	0
10	Argentina	2	3	2777	28	2	0	0	3	2	0	0	1	0	1	0	0	blue	0	0

Transformação de Dados (4/13)

- Ex. 5: conhecendo o método `apply()`
 - O método **`apply()`** é o mais utilizado para transformar dados.
 - Uso básico: permitir com que uma função do python seja aplicada de forma automática e, quase sempre, vetorizada sobre uma coluna inteira !!!
 - Inspiração na programação funcional (ver última aula)
 - O resultado será usado para substituir a coluna original ou gerar uma nova coluna.

```
import numpy as np
flags2 = flags.copy() #clona a base original

# converte o nome para maiúsculo (substitui a coluna)
flags2['name'] = flags2['name'].apply(str.upper)

# cria uma coluna chamada "log_area" com o log10 da área
flags2['log_area'] = flags2['area'].apply(np.log10)
```


Transformação de Dados (5/13)

- Saída:

	name	area	log_area
0	AFGHANISTAN	648	2.811575
1	ALBANIA	29	1.462398
2	ALGERIA	2388	3.378034
3	AMERICAN-SAMOA	0	-inf
4	ANDORRA	0	-inf
..	...	...	...
189	WESTERN-SAMOA	3	0.477121
190	YUGOSLAVIA	256	2.408240
191	ZAIRE	905	2.956649
192	ZAMBIA	753	2.876795
193	ZIMBABWE	391	2.592177

[194 rows x 4 columns]

Transformação de Dados (6/13)

- **Ex. 6: discretização (transforma variável contínua em discreta ou categórica)**
 - Você criar a sua própria função e aplica-la com apply.
 - O exemplo a seguir mostra uma função para discretizar a “area”

```
def classe_area(area):  
    if (area < 500): return 'PEQUENA'  
    elif (area >= 500 and area < 1000): return 'MÉDIA'  
    else: return 'GRANDE'  
  
flags2['tipo_area'] = flags2['area'].apply(classe_area)  
  
print(flags2[['name', 'area', 'tipo_area']])
```

Transformação de Dados (7/13)

- Saída:

	name	area	tipo_area
0	AFGHANISTAN	648	MÉDIA
1	ALBANIA	29	PEQUENA
2	ALGERIA	2388	GRANDE
3	AMERICAN-SAMOA	0	PEQUENA
4	ANDORRA	0	PEQUENA
..	...	...	...
189	WESTERN-SAMOA	3	PEQUENA
190	YUGOSLAVIA	256	PEQUENA
191	ZAIRE	905	MÉDIA
192	ZAMBIA	753	MÉDIA
193	ZIMBABWE	391	PEQUENA

[194 rows x 3 columns]

Transformação de Dados (8/13)

- Ex. 7: padronização da escala

- Em qualquer BD é comum que variáveis numéricas estejam representadas em diferentes unidades de medida.
- Por exemplo, na base flags:
 - **population**: milhões de pessoas
 - **area**: milhares de quilômetros quadrados
 - **colours**: unidades (total de cores em uma bandeira)
- Muitos algoritmos exigem com que os atributos numéricos estejam padronizados dentro de uma **faixa de valores comum**, como **[0.0, 1.0]** ou **[-1.0, 1.0]**.
 - **Ex.:** algoritmos baseados em distância e similaridade (*próxima aula !*)
 - Isso é feito para assegurar que números grandes não “dominem” os pequenos.

Transformação de Dados (9/13)

- Ex. 7: padronização da escala

- Vamos padronizar os atributos “population”, “área” e “colours” do BD flags usando a fórmula abaixo:

$$\text{valor_padronizado} = (x - \min(X)) / (\max(X) - \min(X))$$

- **x**: é o valor do atributo X em uma dada observação
 - **min(X)**, **max(X)**: menor e maior valores de X
 - A saída é um valor entre 0 e 1
-
- Criaremos uma função e usaremos apply. Essa função terá 2 parâmetros usados pela fórmula:
 - min_X e max_X
 - Os parâmetros devem ser passados em uma tupla com o uso de **args**.

Transformação de Dados (10/13)

- **Ex. 7: padronização da escala – PARTE 1: resolvendo com o apply()**

[illegible]

Transformação de Dados (11/13)

- Ex. 7: padronização da escala – PARTE 2: visualizando os resultados

```
# países muito populosos
print('- alguns países com área extensa ou muitos populosos:')
str_grandes = "['USSR', 'BRAZIL', 'CHINA', 'USA', 'CANADA', 'INDIA',
               'PAKISTAN']"
df_grandes = flags2.query("name in {}".format(str_grandes))
print(df_grandes[['name', 'area', 'area_padr',
                  'population', 'pop_padr',
                  'colours', 'colours_padr']])

# outros países
print('\n- outros países:')
str_outros = "['PORTUGAL', 'FRANCE', 'CHILE', 'MEXICO', 'JAPAN']"
df_outros = flags2.query("name in {}".format(str_outros))
print(df_outros[['name', 'area', 'area_padr',
                  'population', 'pop_padr',
                  'colours', 'colours_padr']])
```

Transformação de Dados (12/13)

- Saída:

- alguns países com área extensa ou muitos populosos:

	name	area	area_padr	population	pop_padr	colours	colours_padr
23	BRAZIL	8512	0.379966	119	0.118056	4	0.428571
31	CANADA	9976	0.445317	24	0.023810	2	0.142857
37	CHINA	9561	0.426792	1008	1.000000	2	0.142857
81	INDIA	3268	0.145880	684	0.678571	4	0.428571
132	PAKISTAN	804	0.035890	84	0.083333	2	0.142857
183	USA	9363	0.417954	231	0.229167	3	0.285714
184	USSR	22402	1.000000	274	0.271825	2	0.142857

- outros países:

	name	area	area_padr	population	pop_padr	colours	colours_padr
36	CHILE	757	0.033792	11	0.010913	3	0.285714
59	FRANCE	547	0.024417	54	0.053571	3	0.285714
90	JAPAN	372	0.016606	118	0.117063	2	0.142857
112	MEXICO	1973	0.088072	77	0.076389	4	0.428571
139	PORTUGAL	92	0.004107	10	0.009921	5	0.571429

Transformação de Dados (13/13)

- **Ex. 8: criando coluna diretamente derivada de duas outras**

- Se a derivação vem de um cálculo simples, a criação do campo derivado é trivial (nem precisamos do apply).
- Ex.: densidade demográfica = população ÷ area

```
# densidade populacional (D) = populacao / area
# Na flags original:  area: in thousands of square km;
#                    population: in round millions

flags2['D'] = (flags2['population'] *1e6) / (flags2['area'] *1e3
)

#agora vamos imprimir só os dados do Japão e da Noruega!
#(lembrando que eu havia convertido os nomes para maiúsculo antes)
filtro = (flags2['name']=='JAPAN') | (flags2['name']=='NORWAY')
print(flags2.loc[filtro, ['name', 'area', 'population', 'D']])
```

	name	area	population	D
90	JAPAN	372	118	317.204301
130	NORWAY	324	4	12.345679

Link para código com os exemplos da aula

- Data wrangling – limpeza e transformação de dados
 - https://colab.research.google.com/drive/1dDyHyRHea6XJXNE_-ij3VNmXSwCpsGNL?usp=sharing

EXERCÍCIO

- O DataFrame 'pme', mostrado abaixo armazena dados de 6 pessoas entrevistadas por uma pesquisa hipotética

atributos: renda em salários mínimos, quantidade de empregos, sexo e escolaridade)

- Observe seu formato e dados e escreva o código capaz de resolver os itens abaixo

	<i>renda</i>	<i>empregos</i>	<i>sexo</i>	<i>escolaridade</i>
[0]	6,46	1	F	pós-graduação
[1]	1,50	1	M	fundamental
[2]	0,00	0	F	médio
[3]	2,57	1	M	médio
[4]	9,90	2	M	superior
[5]	6,22	3	F	médio

- Crie o DataFrame acima dando o nome 'pesq' (dica: ver o exemplo no slide 33)
- Gere o dataframe d1 projetando as colunas 'escolaridade' e 'renda'
- Gere o dataframe d2 contendo apenas as linhas referentes a quem tem mais de 1 emprego.
- Gere o dataframe d3 projetando as colunas 'renda', 'empregos' e 'sexo', mas filtrando apenas os entrevistados com nível médio.
- Gere uma nova coluna chamada "muitos_emps" em "pesq", que deve conter o valor "Sim" para quem tem 2 ou mais empregos e o valor "Não" para quem tem 1 ou nenhum emprego.

Referências

HAN, J.; KAMBER, M. PEI, J. Data preprocessing. *In*: HAN, J.; KAMBER, M. PEI, J. **Data Mining: Concepts and Techniques**. 3. ed. Waltham: Morgan Kauffman, 2024. cap 3, p. 83 – 115.

MCKINNEY, W. Python for Data Analysis: Data Wrangling with Pandas, NumPy and Jupyter. 3. ed., [s:]: O'Reilly, 2022.

pandas, <https://pandas.pydata.org/>, 2025. Acesso em: 07 abr. 2025.

RUIZ, A. The 80-20 Data Science Dilemma. **InfoWorld**. [s:], 2017. Disponível em: < <https://www.infoworld.com/article/3228245/the-80-20-data-science-dilemma.html> >. Acesso em: 07 abr. 2025.